

SolidSessionBench: Realistic Query Sequences for User-Oriented Decentralized Environments

Ruben Eschauzier¹[0000–0002–6475–806X] and Ruben Taelman¹[0000–0001–5118–256X]

Department of Electronics and Information Systems, Ghent University – imec

Abstract. To build user-facing applications on decentralized data, client-side decentralized query engines offer a natural integration point. However, optimizing queries in these environments is challenging due to a lack of prior knowledge regarding data distribution and availability. By repeatedly interacting with the same engine from the same user, applications allow the engine to overcome this knowledge gap by exploiting patterns in previously issued queries. However, evaluating these techniques accurately is difficult because public query logs are scarce, and existing benchmarks rely on static, template-based structures that fail to capture realistic behavioral variability. To fill this gap, we introduce an evidence-based simulation of client query usage patterns as a reusable benchmark enabling realistic and reproducible evaluation of client-side optimization techniques. We demonstrate its utility by evaluating client-side caching strategies for link traversal query processing. By comparing our sequence-based approach against traditional, non-sequence-based workloads, we highlight how realistic sequence simulation heavily influences performance evaluations. Ultimately, our benchmark reveals important performance dynamics, such as the impact of session switching and query refinement, that traditional workloads do not. We conclude that modeling realistic user interactions provides an essential foundation for accurately evaluating client-side optimization techniques in decentralized environments.

Keywords: Link Traversal-based Query Processing · Benchmarking · Workload Generation · Caching.

RT: Important: double-check in detail the resource cfp. For example, I can't find the persistent URI, canonical citation, and license specification (typically all mentioned after the abstract). There may be other things missing as well.

1 Introduction

Emerging web applications increasingly rely on decentralized architectures to give users control over their personal data, requiring query engines to dynamically fetch and integrate information distributed across the web. While centralizing this data simplifies query processing, it often conflicts with privacy

requirements and fine-grained access controls. Structured decentralized ecosystems such as Solid [46] and Mastodon [43] encourage users to keep their own data in small, independent stores that follow shared data models. This approach strengthens user autonomy, yet it makes query execution more difficult because engines must identify and query many heterogeneous data sources, each with its own access policies.

Traditional federated querying approaches [49,47,30] support decentralization of sources, but typically assume a small (10-100) set of well-described and consistently available endpoints [16]. These assumptions fail once data is spread across thousands of small units that enforce local access-control policies. In addition, enforcing fine-grained access control policies [20] on large sources with expressive interfaces introduces notable overhead [40], making standard approaches unsuitable for highly decentralized environments.

Newer decentralized environments, such as Solid, often expose data as lightweight HTTP resources instead of highly expressive query interfaces. Many of these resources are not known in advance and can only be discovered during query execution. Their simplicity lowers the cost of enforcing fine-grained controls. This setting requires a federation model that can operate over a very large number of small sources, many of which may be encountered at runtime, without relying on prior aggregation. Traditional federated approaches are insufficient in this setting due to the assumption of a small number of large endpoints.

Executing SPARQL queries using Link Traversal-based Query Processing (LTQP) [28] fits these needs. It discovers documents incrementally by following URIs encountered during execution, starting from an initial set of seed documents provided by either the user or the query. Its link-driven, asynchronous traversal supports fine-grained permissions and avoids the requirement to know the entire data distribution beforehand.

However, the fact that LTQP discovers documents incrementally creates significant challenges for efficient execution. Using the traditional optimize-then-execute paradigm is difficult, as the query itself determines which data will be accessed. As a result, the optimizer has no prior knowledge of the relevant data until the query’s traversal has already begun.

Some approaches optimize query execution without prior knowledge by using adaptive query processing [25] and zero-knowledge query planning [27]. These yield modest but inconsistent performance gains, depending on the decentralized environment and the heuristics used [53]. However, other aspects of zero-knowledge LTQP optimization, such as link prioritization [29], are unlikely to improve query result arrival times in emerging decentralized environments [19].

Consequently, other works explore using precomputed server-side information as prior knowledge or relevance indexes to efficiently execute LTQP queries [54,55,24]. While these approaches offer greater performance benefits, they require the server to maintain these indexes. This raises the cost of serving resources and complicates privacy and access control.

To avoid server-side overhead, engines can derive prior knowledge directly on the client. In decentralized settings, LTQP is inherently client-side, with

a single engine instance serving one or more applications. As these applications issue sequences of correlated queries, the engine identifies frequently accessed data and its characteristics. This allows engines to implement *personalized* query optimization algorithms that leverage local access patterns to build necessary prior knowledge. Benchmarking these approaches requires realistic query sequences. Traditional benchmarks (e.g., WatDiv [1], BSBM [10]) execute isolated or repeated query templates, which fails to capture client-side dynamics accurately. Furthermore, while centralized SPARQL optimization relies on real query logs [9,42,11,50], such logs do not yet exist for decentralized LTQP environments. Consequently, we need a standardized benchmark that realistically simulates user query sequences to prevent fragmented evaluation practices.

In this paper, we address this gap in existing benchmarks by introducing SolidSeqBench, a new benchmark that builds upon SolidBench [53] to generate realistic query sequences. These sequences capture three usage patterns identified in social network application literature, the domain simulated by SolidBench, alongside patterns observed in real-world SPARQL logs. Consequently, our benchmark enables researchers to evaluate personalized query optimization techniques against the actual user behaviors they will encounter in real client-facing applications.

To demonstrate our benchmark’s utility, we implement three baseline client-side caching strategies in the LTQP engine Comunica [51,53]. We use these intentionally simple algorithms to establish a performance baseline and highlight the necessity of sequence-based evaluation.

2 Literature Review

2.1 Existing Benchmarking Approaches

For the evaluation of client-side benchmarking techniques that rely on preceding queries, we primarily investigate caching literature, as this is a well-studied problem relevant to our work.

In the literature, we find three distinct benchmarking approaches: simulated micro benchmarks, simulated end-to-end query benchmarks, and real-world query logs.

Simulated micro benchmarks isolate specific execution variables, such as cache size [26] or indirectly, cache hit rate [38]. While useful for analyzing isolated mechanisms, they provide an incomplete picture of performance under realistic, dynamic workloads.

Simulated end-to-end benchmarks evaluate overall behavior using query sequences. However, these sequences are typically constructed by randomly grouping isolated queries [39,10,26,38,22], applying simple statistical distributions like Pareto [57,34,6], or relying on narrow application scenarios [14,34] with custom build query sequences. Each of these methods suffers from distinct limitations. Random grouping ignores real-world behavior entirely. Simple statistical distributions are too basic to capture complex user habits. Finally, static scenarios

generate too few queries to provide comprehensive benchmark coverage. Consequently, none of these approaches simulate how users actually explore data, missing how they group related queries into short sessions and constantly adjust their search parameters based on immediate results.

Real-world query logs provide the gold standard for evaluation. While extensive logs are widely used to benchmark centralized SPARQL endpoints [45,50,9,58,41,32], none currently exist for decentralized LTQP environments.

To avoid the fragmented evaluation practices of current simulated approaches, the next best option is a unified simulated benchmark. A shared benchmark would provide a common basis for comparison, improve reproducibility, and prevent each paper from having to create its own evaluation methodology.

2.2 Real-world Query Usage Patterns

To inform the benchmark design on what patterns should be simulated, we will outline three relevant client query usage patterns observed in the literature. The relevance is determined for our social network application use case.

User Interest-Based Query Behavior Work on social networks shows users form communities based on shared interests [36,15,7,31]. These networks exhibit power-law degree distributions, small-world structures, and scale-free properties. This topology creates a dense core of high-degree nodes that connect many smaller, strongly clustered groups [36]. Simulation studies confirm that this interest-driven group formation is stable and reliably modeled [2,23].

Furthermore, empirical research formally establishes that shared interests directly drive a higher degree of explicit and implicit interactions between users [37]. These findings justify our simulation of user-relatedness and interest-driven behavior when generating query sequences.

Logical Query Sessions Analysis of web browsing behavior shows that activity is best grouped into *logical sessions*: sequences of user interactions focused on specific goals, rather than purely time-bound intervals [35,44,48]. These task-oriented sessions frequently involve non-linear navigation, including backtracking. New sessions begin in approximately 15% of clicks, following a log-normal distribution with average size and depth means of 6.1 and 3, respectively.

These patterns also appear in application navigation. In a decentralized social media application, for example, a user viewing a feed might click a creator’s username, view comments, or like a post. Because the parameters of each new query derive directly from the preceding query’s results, these queries chain together into structures mirroring logical web sessions. Additionally, direct searches for specific posts or topics reflect standard session-switching behavior.

Consequently, interactive applications generate sequences of interdependent requests [56]. This dynamic informs the design of Facebook’s TAO data store, which is explicitly optimized for workloads driven by the user’s step-by-step traversal of the social graph within the interface [13].

Query Refinement DBpedia SPARQL logs show users frequently submit related queries in streaks. In these streaks, queries share an underlying goal and are iteratively refined to achieve this objective [12,59]. While early session studies

focused on robotic traffic [59,12], our benchmark models organic sequences generated through software interactions [59,33]. Because application design, rather than underlying data organization, drives these organic queries, we expect these patterns to also exist in decentralized environments. We exclude robotic queries from our benchmark, as these rely heavily on templated query executions that traditional benchmarks can already simulate [53,10,1]. Finally, across all query logs, analysis of total usage and reformulations shows that adding or removing FILTER, UNION, and OPTIONAL operators drives the vast majority of reformulations.

Recent analyses of Wikidata query logs further demonstrate that query development is an iterative process of design, debugging, and maintenance [5]. Based on this empirical data, exploratory querying encompasses six core behaviors [5]: **result refinement** to reduce records via filters or selective triple patterns, **result generalization** to retrieve more entities by broadening parameters, such as adding superclasses or increasing limits, **result extension** to fetch additional attributes by appending new triple patterns, **bug fixing** to correct structural errors like invalid URIs, **query refinement** to adjust complex language features, and **shifting query intent** to pivot topics by altering core predicates.

These findings confirm that real-world querying relies on dynamic, step-by-step modifications rather than the static execution of independent templates. Consequently, existing evaluation frameworks fail to capture how users actively interact with data systems. Evaluating system performance for these exploratory query sequences remains a critical research challenge, explicitly requiring new benchmarks to validate progress [5]. Therefore, modeling these empirical sequence patterns is essential to accurately assess client-side query optimization techniques.

3 Query Sequence Benchmark

In this section, we detail our simulation methodology and the resulting hyperparameters. We base our sequence generator on SolidBench [53], which simulates realistic decentralized social media applications within the Solid ecosystem. SolidBench utilizes the Social Network Benchmark (SNB) dataset [18,3] and applies the Linked Data Benchmark Council (LDBC) choke point methodology [4]. Its workload combines standard SNB *interactive* queries with custom *discover* templates, which explicitly target Link Traversal Query Processing (LTQP) choke points in decentralized environments.

To support sequence-based evaluation, we extend three core components of the SolidBench ecosystem:

- **Query Generator:** We modify the generator to produce correlated query sequences that reflect empirical real-world usage patterns.
- **Dataset Generator:** We enhance the data generation process to model interest-based similarities between simulated users.

- **Benchmark Runner:** We integrate sequence execution logic into the existing runner to facilitate straightforward, reproducible sequence-based experiments and allow easy generation of default configuration files.

3.1 Simulation Methodology

Our methodology models three interdependent behaviors: task-oriented logical sessions, user interests for data locality, and refinement patterns for iterative query development.

Logical Query Sessions To model logical sessions, we categorize queries into four primary topics: person-specific messages, forum information, person attributes, and message attributes. Because queries in decentralized environments often depend on preceding results, we map the output types of each template to valid subsequent templates. This creates a directed graph of query progression that governs the flow of a logical session.

Within a session, a template’s *instantiation values* are typically derived from the previous query’s results. To realistically simulate this “click-through” behavior, we execute centralized equivalents of the LDBC SNB queries using QLever [8]. We then randomly sample the returned result set to instantiate the subsequent query template. If a query yields no results, we fall back to our default, interest-based instantiation methodology.

Finally, we simulate logical session switching. Drawing from web browsing behavior, we assign each sequence a session-switching probability sampled from a log-normal distribution with a mean of 15% [35,48]. During sequence generation, a session switch triggers a random choice with equal probability: the engine either starts a new session (using interest-based instantiation) or resumes a previous one (deriving instantiation values from that session’s last executed query). To prevent repetitive queries and maintain a uniform distribution of template instantiations, we explicitly prohibit backtracking within our model. Additionally, we maintain a balanced template distribution by dynamically scaling down each template’s selection probability proportionally to its overall frequency.

User Interest Modeling Individuals interact more frequently with users sharing similar interests [37]. To simulate this behavior, we use the underlying LDBC dataset utilized by SolidBench [18], which inherently models realistic user preferences. The LDBC generator, Datagen, produces person profiles containing specific interests, educational backgrounds, and workplaces. It links users based on these demographic and interest overlaps, yielding a non-random network topology with a realistic degree distribution.

We use this interest-driven topology to generate template instantiation values for the initial query of each logical session. (As established, subsequent queries derive their values from preceding result sets). Specifically, we construct a binary feature vector v_k for each user k : $v_k = (x_{k,1}, \dots, x_{k,n}, y_{k,1}, \dots, y_{k,m})$, where

$x_{k,j} = 1$ if user k holds stated interest j , and $y_{k,l} = 1$ if user k completed study l .

For each sequence, we randomly select a base person k and compute the cosine similarity between v_k and the feature vectors of all other users in the dataset.

We then select template instantiation values probabilistically based on these similarity scores by applying a softmax function with a temperature parameter T :

$$P(c_{i,k}) = \frac{e^{s_i/T}}{\sum_{j=1}^N e^{s_j/T}}, \quad (1)$$

where s_i represents the similarity score for candidate person i . To minimize computational overhead at large scale factors, we restrict this softmax operation to the top N candidate entities (ranked by similarity) rather than the entire population.

Finally, for query templates requiring non-person instantiation values (such as posts or activities), we compute selection probabilities based on the similarity scores of the users who created that content.

Refinement Patterns The benchmark simulates query refinement by randomly applying composable transformations to an instantiated query template. Following our literature review, we specifically focus on transformations to add or modify BGPs, OPTIONALs, UNIONs, and query instantiation values.

To ensure these transformations meaningfully alter query execution characteristics, rather than superficially appending triples that maintain a one-to-one relation with the result set, we apply a choke point-based design methodology [4]. Specifically, we define several planning choke points for client-side query optimization in decentralized environments, building on the original SolidBench choke points [53]: **P.1** involves adding or removing a **FILTER**, which requires prior knowledge to adjust query plans and estimate selectivity; **P.2** addresses adding or removing **UNION** or **OPTIONAL** clauses to alter filter push-down capabilities; **P.3** covers adding or removing selective triple patterns; and **P.4** focuses on adding or removing non-selective triple patterns.

Beyond planning, these refinements also impact traversal processes. We identify the following traversal choke points: **T.1** expands or contracts the reachable sub-web through predicate additions; **T.2** shifts the sub-web by substituting the traversal starting point; **T.3** changes the number of hops required to obtain relevant data, corresponding to SolidBench choke points L.1–L.6 [52]; and **T.4** alters the usability of structural indexes, such as type indexes, corresponding to SolidBench choke point L.8 [52].

The generator applies these refinements sequentially to a base SolidBench query template. By default, every refinement pattern includes a reciprocal “undo” operation. The supported refinement operators include:

- **OPTIONAL and UNION blocks:** Adding new blocks or inserting triples into existing ones (including specific **UNION** branches).

- **Basic Graph Patterns (BGPs):** Adding new triple patterns to existing BGPs; the sets of triple patterns that form the core graph-matching logic of a SPARQL query.
- **FILTER statements:** Appending new query constraints.
- **Value substitution:** Replacing template instantiation values to start traversal from a different seed document.

The generator also supports variable instantiation within refinements, allowing users to bind pattern variables to specific values randomly selected from candidates provided through configuration files. Users can define custom query refinement patterns using JSON files to augment the provided default configurations. These defaults, along with documentation and pattern descriptions, are available on GitHub.¹

Through this flexible generator, we explicitly simulate the empirical exploratory usage patterns [5] identified in Section 2.2. We map these empirical patterns directly to our composable query transformations and choke points:

- **Result Refinement:** We simulate this by adding or removing **FILTER** constraints and selective triple patterns in the BGP to adjust query selectivity (Choke points P.1, P.3).
- **Result Generalization:** We replicate this by adding new triples to existing BGPs (Choke points P.3, P.4), mirroring how users expand queried classes and fetch related entities.
- **Result Extension:** We simulate this by adding triple patterns to BGPs, **OPTIONAL** blocks, and **UNION** blocks (Choke points P.3, P.4, P.2).
- **Query Refinement:** We model this through the addition or removal of **OPTIONAL** and **UNION** blocks (Choke point P.2).

As our refinement rules are fully composable, the generator naturally reflects the empirical observation that exploratory patterns are not mutually exclusive and frequently co-occur within a single session.

3.2 Hyperparameters

The sequence generation process introduces several new hyperparameters, which are detailed in Table 1 together with their default values. To guarantee reproducibility despite extensive random sampling, a single seeded pseudo-random number generator (PRNG) is used throughout the entire pipeline.

¹ <https://github.com/RubenEschauzier/SolidBench.js/tree/feature/user-based-query-sequences/templates/refinements>

Table 1. LogT-normal distribution parameters (μ, σ) control the queries per sequence, queries per logical session, and the probability of switching logical sessions. Refinement parameters define the likelihood (`patternProbability`) and bounds (`min/maxRefinementLength`) of generating a refinement sequence for a given query. Instantiation relies on a softmax `temperature` and a `maxLogits` cutoff for similarity-based entity selection.

Category	Parameter	Value
Distributions (μ, σ)	Sequence Length	3.0, 0.2
	Session Length	1.7, 0.5
	Transition Probability	-2.0, 0.25
Refinements	<code>patternProbability</code>	0.1
	<code>min/maxRefinementLength</code>	2, [3–5]
Instantiation	<code>temperature</code>	0.1
	<code>maxLogits</code>	5–10

4 Caching in Link Traversal-based Query Processing

To make this work self-contained, we first briefly outline the mechanics of Link Traversal-based Query Processing (LTQP). Unlike traditional query engines that evaluate queries against a centralized, pre-built index, LTQP evaluates queries directly over the live Web using the “follow-your-nose” principle of Linked Data. Execution relies on two core components: a *link queue* and an *active local store*. The engine initializes the link queue with seed URIs (often extracted from the query). It then iteratively dereferences these URIs, retrieving RDF documents. The triples from these documents are deposited into the active local store; a temporary, query-specific, in-memory dataset over which the SPARQL operators are actually evaluated. Simultaneously, new URIs are discovered within the retrieved data and added to the link queue according to specific *reachability semantics*, which dictate which links are relevant to follow based on the query structure. Furthermore, to optimize memory usage, engines like Comunica [51] typically employ *dictionary encoding* within the local store, converting lengthy string-based URIs and literals into compact integer identifiers.

Hartig [26] originally established the fundamental concept of caching within this paradigm, demonstrating that keeping a local repository of recently accessed Web documents mitigates the high network latency inherent to LTQP.

To validate the query sequences generated by the SolidBenchSequence (SBS) benchmark, we implemented three caching approaches in the Comunica link traversal engine [51,53]. Specifically, we cache the triples from documents dereferenced during link traversal in-memory. High cache hit rates eliminate redundant HTTP requests, increasing traversal speed and reducing query latency. To evaluate how different cache management strategies impact this latency, we compare the following three configurations:

RT: At this point, I realized that you never actually explain how LTQP works (link queue, reachability semantics, ...). Some minimal explanation of this

must be added, to make this work self-contained, to make it possible for readers not knowing about LTQP to understand this paper. For example add this in the introduction or literature review. **RT:** RE: Mention dictionary encoding in the explanation and then refer back to this to explain why indexed cache will use less memory compared to unindexed

Unindexed Cache This baseline approach caches the triples obtained during traversal. Upon a cache hit, these quads are retrieved and indexed into the active local store **RT:** Unclear to the reader what the "active local store" is. This for example should be explained in the general description of LTQP. for query execution. **RT:** Do I understand it correctly that this approach is essentially just an array of quads/triples?

Indexed Cache Alternatively, we index triples before caching them. Upon a cache hit, the engine extracts only triples matching the quad patterns in the current query. While this reduces the memory footprint and indexing overhead of the local store, as triples are filtered before being added to the store. **RT:** It's unclear why this reduces memory usage. The same data is stored, right? Just in a different way?, it increases the initial cache ingestion cost. We hypothesize this trade-off is only beneficial given sufficiently high hit rates.

Indexed Cache + Cardinality estimation Because the cache is indexed, it can execute count queries to estimate quad pattern cardinalities. Traditional LTQP relies on zero-knowledge heuristics due to a lack of prior knowledge [27]. By estimating cardinalities against *all* cached documents, regardless of their reachability for the current query, we can replace these heuristics with Comunica's default cost-based optimizer [51] to generate more accurate query plans.

RT: It may be helpful for the reader to visually represent these 3 approaches next to each other.

For each caching approach, we investigate three sizes **RT:** I would emphasize here again that you focus on in-memory storage: small (350,000 quads), medium (700,000 quads), and large (1,400,000 triples). These triple counts correspond to about 10%, 20%, and 40% of the total generated dataset size.

Our implementations adhere to the RFC-9111 specification [21], meaning the cache correctly interprets standard HTTP headers (like Cache-Control) to determine how long to store data, just like a web browser. However, because the benchmark dataset is static, cache entries require no revalidation and never become stale. Consequently, the observed hit rates are likely higher than in a live environment. Furthermore, while the fundamental concept of caching in link traversal is established [26] **RT:** This hartig2011caching paper should really be discussed in more detail. Perhaps you can add it to this dedicated LTQP description I asked for above?, integrating and comparing these distinct caching strategies provides novel and strong baselines for evaluating caching in traversal engines, upon which future research into advanced caching approach can build.

5 Evaluation

We evaluate our benchmark using a scale 0.1 SolidBench dataset (158,233 RDF files, 1,531 vaults, 3,556,159 triples). Using the default hyperparameters in Table 1, we generate 8 sequences totaling 192 queries. These queries are from the interactive workload, are split into discover and short queries, with multiple templates belonging to either of these groups. Experiments run on an Ubuntu 20.04 machine (Intel Core i5-9400) with 16 GB RAM allocated to the engine and a 180-second query timeout. We repeat each sequence 10 times to minimize variance and inject a uniform 70–90 ms latency to simulate a realistic network environment. The cache is scoped for a sequence, meaning no cache-state is shared between different executions or repetitions of sequences.

These experiments serve two goals. Primarily, we validate the sequence generation by assessing how our three design choices impact query correlation, measured via cache *hit rates* and *evictions*. Secondly, we establish a caching performance baseline for a LTQP engine. While we report execution speedups to demonstrate the benchmark’s utility, our focus is on validating the benchmark design rather than exhaustively evaluating the caching algorithms.

RT: Can you briefly hint to the subsections that will come hereafter? (so the reader knows what to expect)

5.1 Global Cache Performance

Figure 1 compares performance profiles [17] for each approach against the no-cache baseline. As expected, all caching methods outperform the baseline.

For the unindexed cache, small (350k triples) and medium (700k triples) sizes perform similarly. The large cache (1,400k triples) significantly improves speed by retaining enough entries across the sequence. However, the baseline resolves more total queries, suggesting cache management overhead causes timeouts for longer queries.

The indexed cache has a higher failure rate, as initial indexing costs slow execution when the eviction rate outweighs the increased efficiency of cache hits. Still, it accelerates successful queries more than the unindexed cache. It also reaches critical capacity sooner, with medium and large sizes performing similarly.

Overall, the indexed cache with cardinality estimation performs best, increasing speed and solving *more* queries than the baseline. Notably, larger cache sizes degrade its performance. Indicating that cardinality estimation favors smaller, fresher caches, as it evaluates all entries rather than only traversal-relevant ones. **RT:** This is surprising. I would expect larger caches to provide more accuracy for cardinality estimation. Not sure what to do with this surprising fact besides just reporting though. Maybe you can double-check if it’s really correct?

5.2 Logical Sessions

To evaluate the impact of logical sessions on caching performance, we analyze the hit rate deviation (Δ) from the overall average when queries initiate a new

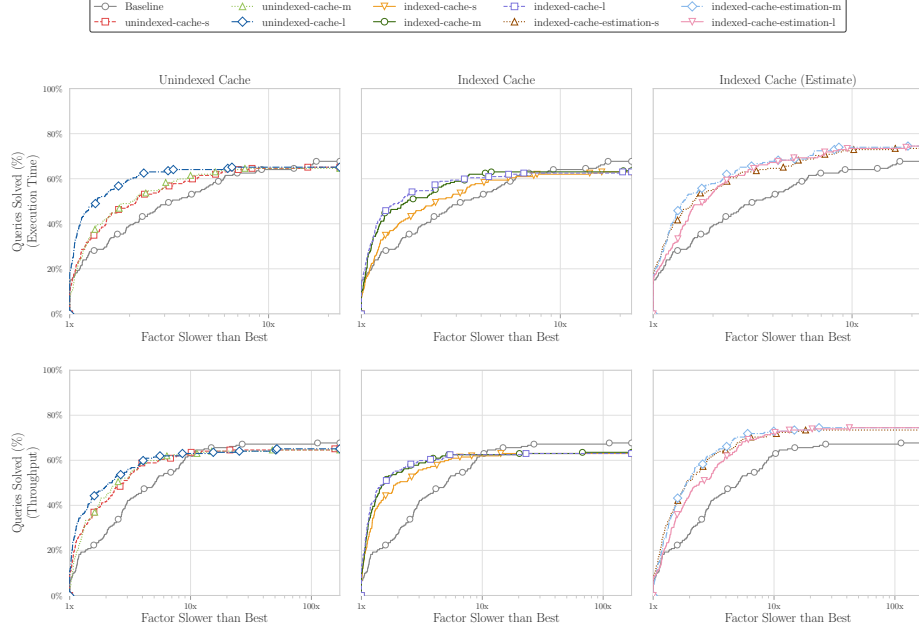


Fig. 1. Performance profiles of the tested cache algorithms. The y-axis shows the percentage of queries executing within a factor (x-axis) of the fastest algorithm. Curves closer to the top-left indicate better overall performance. The y-intercept ($x = 1$) shows how often an algorithm is the outright fastest, while the rightward trajectory demonstrates robustness (e.g., $x = 2, y = 80\%$ means 80% of queries execute within twice the fastest time). From the figure, we observe that all caching-based approaches outperform the no-cache baseline.

session, switch to an existing one, or remain in the current session. Table 2 shows that new sessions severely degrade hit rates, existing session switches cause moderate degradation, and intra-session queries improve performance. These effects are strongest in small and medium caches.

Consequently, optimal client-side cache sizing depends heavily on user session-switching behavior. These findings highlight the need for eviction policies that better retain cross-session entries and validate our simulation design; ignoring session dynamics would obscure critical real-world performance shifts.

Future work should investigate whether query sequences should share a frequently accessed core of cache entries. Protecting these entries from eviction could mitigate the performance penalties of session switching. **RT: Or: separate caches per sessions need to be kept (which requires engines to identify queries being within a session)**

Table 2. Impact of logical session status on algorithm performance. Values represent absolute and percentage cache hit rate deviations from the global template average across new, existing, and within-session queries. **RT: Mention clearly if negative values are good or bad.**

Cache Algorithm	New Session		Existing Session		Within Session	
	Abs.	%	Abs.	%	Abs.	%
unindexed-l	-0.066	-12.278	-0.014	-2.474	0.011	1.931
unindexed-m	-0.056	-15.649	-0.031	-7.900	0.024	5.486
unindexed-s	-0.071	-24.275	-0.021	-6.551	0.017	4.582
indexed-estimate-l	-0.053	-10.573	-0.015	-2.796	0.012	2.127
indexed-estimate-m	0.028	7.174	-0.000	-0.064	-0.000	-0.035
indexed-estimate-s	-0.044	-14.305	-0.013	-3.921	0.010	2.924
indexed-l	-0.026	-5.011	-0.003	-0.571	0.003	0.467
indexed-m	-0.087	-22.782	-0.021	-5.098	0.017	3.682
indexed-s	-0.033	-11.036	-0.015	-4.470	0.012	3.153

5.3 Refinement Patterns

To evaluate iterative query refinement simulation, we analyze cache behavior across refinement depths. Figure 1 shows that hit ratios increase progressively as queries undergo sequential refinement. Across all algorithms, hit rates increase at depths 2 and 3, confirming that refinement heavily reuses prior data.

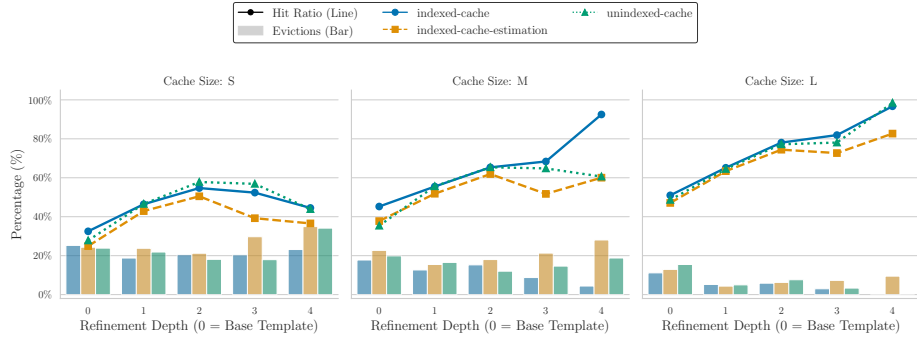


Fig. 2. Cache hit ratios and percentage of cache capacity evicted across refinement depths for small, medium, and large cache configurations. **RT: I would recommend using "Cache hit rate (%)" on the Y axis**

At higher refinement depths, performance depends heavily on cache capacity. Large caches sustain high hit rates through depth 4, whereas small caches experience significant thrashing, and degraded hit ratios.

Surprisingly, the indexed cache without estimation maintains high hit rates even with smaller capacities. This counterintuitive result stems from implementation differences that influence how the single-threaded Node.js event loop schedules asynchronous traversal and join executions **RT**: In order to understand this sentence, we definitely need this high-level description of LTQP, and how it's implemented in Comunica.. This scheduling variance alters traversal order and rate, ultimately impacting cache eviction and hit performance. **RT**: I understand the variance, but from this text, I don't understand why this leads to higher hit rate with smaller capacities.

The observed cache hit rate and eviction differences directly translate to measurable execution speedups. Figure 2 illustrates that the geometric mean speedup generally scales positively with refinement depth, with a more pronounced effect for the throughput. The indexed-cache algorithm proves particularly effective for prolonged exploratory sequences, achieving substantial throughput gains at depth 4 compared to the unindexed alternatives.

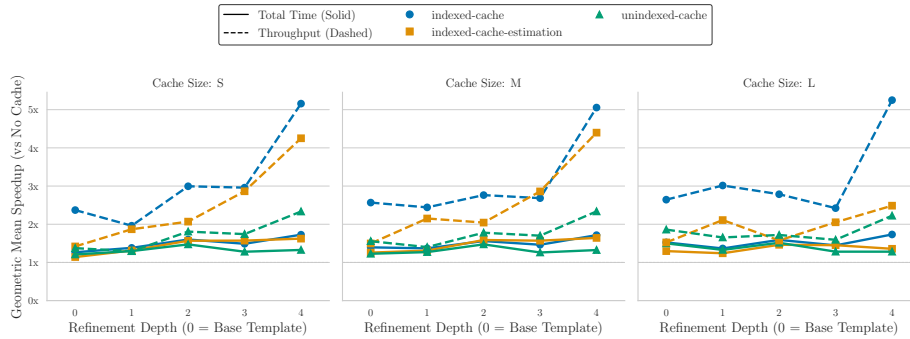


Fig. 3. Geometric mean of observed speedups and increases respectively for total execution time and query throughput relative to the no caching baseline across refinement depths.

5.4 Comparison To Other Evaluation Methods

Microbenchmarks - Theoretical Hit Rate Analysis Cache literature extensively uses microbenchmarks of various forms to analyze caching performance [38,26]. To compare the conclusions of such a benchmark to the conclusions of the benchmark introduced in this paper we investigate theoretical caching speedup without management costs, by executing each query template twice with simulated cache miss probabilities. We excluded the estimation-based approach, as using the full cache for estimation would skew the results for miss rates above zero.

Figure 4 shows representative performance profiles for the workload. Most templates (8 out of 12) show reliable improvement that scales with the hit rate

for both algorithms. However, certain long-running queries (e.g., interactive-discover-6, 7, and short-3) show no speedup, and some even experience slowdowns. These templates traverse many solid pods but do not require all traversed data. We hypothesize that rapidly serving documents from an in-memory cache overwhelms the system with data, slowing result production. Conversely, HTTP network latency gives the JavaScript event loop time to process the join execution pipeline, yielding initial results faster since only the first few requests are needed. **RT: This last sentence is clear to me, but is another example that requires more background in a dedicated Comunica LTQP description**

Both caching approaches perform similarly overall, but the indexed cache significantly reduces time-to-first-result across most templates. As expected, indexed caching is highly advantageous without evictions. However, the initial indexing cost creates a trade-off: it amplifies both speedups and slowdowns relative to an unindexed cache. While this micro-benchmark approach aligns with prior literature [38], our mixed findings show that micro-benchmarks alone fail to thoroughly evaluate client-side caching optimizations. This limitation demonstrates the clear need for the comprehensive benchmark we introduce in this work.

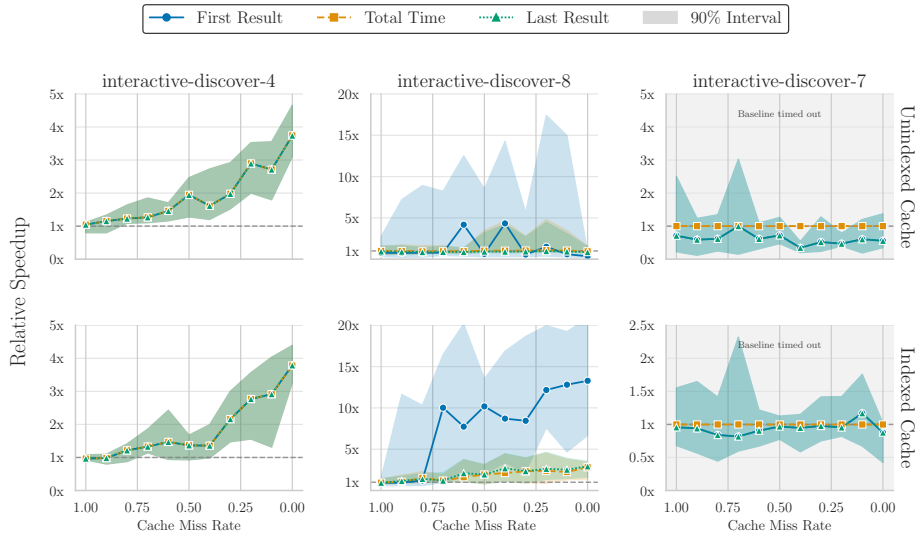


Fig. 4. Performance of unindexed and indexed caches across representative query templates. The y-axis shows the mean speedup relative to a baseline with a 0% hit rate. Shaded regions denote the 90th percentile **RT: Why is the third caching approach not mentioned? If irrelevant, make sure to explain why.**

Random Query Sequences Another common approach is to use random query sequences from an existing benchmark [39,10,26,38,22]. Figure 5 shows execution

times for sequences created by randomly grouping default SolidBench.js queries. For these random sequences, only large caches perform well, while smaller ones degrade below the baseline. These results misleadingly suggest that large caches are vital, whereas our benchmark demonstrates significant performance benefits even with smaller sizes. As expected, the estimation cache performs well in both scenarios because cardinality estimation does not require cache hits.

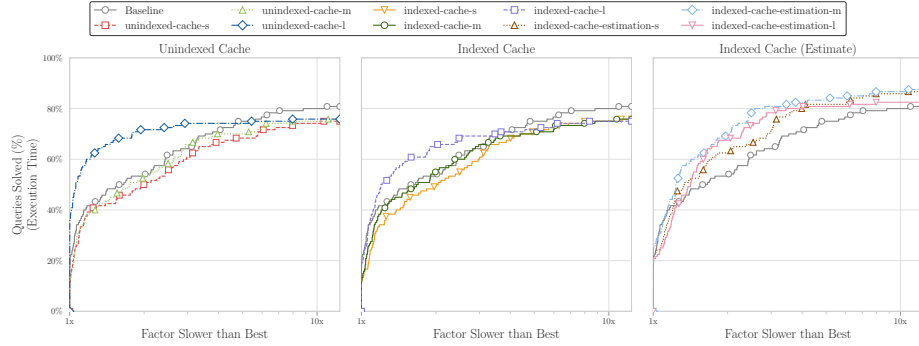


Fig. 5. Performance profile of the tested cache algorithms on random query sequences. The y-axis shows the percentage of queries that execute within a given performance factor (x-axis) of the fastest algorithm for that query.

Scope and Limitations We evaluate SolidSessionBench against microbenchmarks and random sequences, but explicitly exclude hand-crafted scenarios and simple distributions. Hand-crafted methods lack scalability, while simple distributions fail to capture complex user habits. By automating logical sessions and user interests, our benchmark functions as a scalable superset of both approaches, making direct comparison redundant. By contrast, the compared methods do not incorporate these sequence attributes, and our results show this omission risks drawing inaccurate conclusions.

6 Conclusion

In this work, we introduced SolidSessionBench, an evidence-based benchmark that generates realistic query sequences for decentralized environments. By explicitly modeling logical sessions, user interests, and iterative query refinement, our benchmark captures critical client-side performance dynamics that traditional, isolated query workloads obscure. Our evaluation of caching strategies confirms that integrating these empirical sequence behaviors is essential to accurately assess client-side optimizations.

RT: Since reviewers may ask about this, let’s briefly mention the work done around Linked Web Storage. (either here, or earlier in the paper) I’d say that

this benchmark uses Solid as a use case, but the underlying concepts are general, and will directly apply to LWS as well (once the working group is ready). You can say that we will provide an LWS-representation once the LWS specs are stable.

To guarantee long-term maintenance of the benchmark, we are actively integrating SolidSessionBench into the core SolidBench repository via open pull requests. By tying this benchmark to the established SolidBench ecosystem, we use its multi-year track record of reliable maintenance and community support. All source code, data, and associated experiment setups are publicly available at <https://github.com/RubenEschauzier/simulated-query-sequence-paper>.

{TODO: Depending on status of PRs either mention its already integrated or still WIP (Can do this through the url) **}**

RT: Some things to incorporate based on the resource review criteria:

Is there evidence of usage by a wider community beyond the resource creators or their project? Alternatively (for new resources), what is the resource's potential for being (re)used? -> Explain briefly how SolidBench has been adopted as a proxy for how this resource will be used.

Is the resource easy to (re)use? For example, does it have high-quality documentation? Are there tutorials available? -> Are there docs? Make sure to have clear READMEs, and link to them explicitly.

Is there potential for extensibility to meet future requirements? -> You can probably spend a few words on this as well.

Is there a sustainability plan specified for the resource? Is there a plan for the medium and long-term maintenance of the resource? -> You briefly mention this in the conclusion, but I think it would be stronger if we add an explicit sustainability plan for SolidBench (e.g. in its wiki, similar to how we do it for Comunica), and link to it from the paper.

"Declaration of use of Generative AI" section missing.

References

1. Aluç, G., Hartig, O., Özsu, M.T., Daudjee, K.: Diversified stress testing of rdf data management systems. In: International Semantic Web Conference. pp. 197–212. Springer (2014)
2. Amblard, F., Bouadjo-Boulic, A., Gutiérrez, C.S., Gaudou, B.: Which models are used in social simulation to generate social networks? a review of 17 years of publications in jasss. In: 2015 Winter Simulation Conference (WSC). pp. 4021–4032. IEEE (2015)
3. Angles, R., Antal, J.B., Averbuch, A., Birler, A., Boncz, P., Búr, M., Erling, O., Gubichev, A., Haprian, V., Kaufmann, M., et al.: The ldbs social network benchmark (2020)
4. Angles, R., Boncz, P., Larriba-Pey, J., Fundulaki, I., Neumann, T., Erling, O., Neubauer, P., Martinez-Bazan, N., Kotsev, V., Toma, I.: The linked data benchmark council: a graph and rdf industry benchmarking effort. vol. 43, pp. 27–31. ACM New York, NY, USA (2014)

5. Arenas, M., Franconi, E., Hammerer, J., Hartig, O., Hose, K., Koesten, L., Konstantinidis, G., Libkin, L., Martens, W., Sasaki, Y., et al.: Exploring exploratory querying. *Proceedings of the VLDB Endowment* **18**(13), 5731–5739 (2025)
6. Arnold, B.C.: Pareto distribution. *Wiley StatsRef: Statistics Reference Online* pp. 1–10 (2014)
7. Backstrom, L., Huttenlocher, D., Kleinberg, J., Lan, X.: Group formation in large social networks: membership, growth, and evolution. In: *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 44–54 (2006)
8. Bast, H., Buchhold, B.: Qlever: A query engine for efficient sparql+ text search. In: *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, pp. 647–656 (2017)
9. Berendt, B., Hollink, L., Luczak-Rösch, M., Möller, K., Vallet, D.: Usewod2013 3rd international workshop on usage analysis and the web of data. 10th ESWC-Semantics and Big Data, Montpellier, France (2013)
10. Bizer, C., Schultz, A.: The berlin sparql benchmark. In: *Semantic Services, Interoperability and Web Applications: Emerging Concepts*, pp. 81–103. IGI Global Scientific Publishing (2011)
11. Bonifati, A., Martens, W., Timm, T.: DARQL: Deep Analysis of SPARQL Queries. In: *Companion Proceedings of the The Web Conference 2018*, pp. 187–190. WWW '18, International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE (Apr 2018). <https://doi.org/10.1145/3184558.3186975>, <https://dl.acm.org/doi/10.1145/3184558.3186975>
12. Bonifati, A., Martens, W., Timm, T.: An analytical study of large sparql query logs. *The VLDB Journal* **29**(2), 655–679 (2020)
13. Bronson, N., Amsden, Z., Cabrera, G., Chakka, P., Dimov, P., Ding, H., Ferris, J., Giardullo, A., Kulkarni, S., Li, H., et al.: {TAO}:{Facebook's} distributed data store for the social graph. In: *2013 USENIX annual technical conference (USENIX ATC 13)*, pp. 49–60 (2013)
14. Chatzopoulos, S., Vergoulis, T., Skoutas, D., Dalamagas, T., Tryfonopoulos, C., Karras, P.: Atrapos: Evaluating metapath query workloads in real time. *CoRR* (2022)
15. Clark, A.: Understanding community: A review of networks, ties and contacts (2007)
16. Dang, M.H., Aimonier-Davat, J., Molli, P., Hartig, O., Skaf-Molli, H., Le Crom, Y.: Fedshop: A benchmark for testing the scalability of sparql federation engines. In: *International Semantic Web Conference*, pp. 285–301. Springer (2023)
17. Dolan, E.D., Moré, J.J.: Benchmarking optimization software with performance profiles. *Mathematical programming* **91**(2), 201–213 (2002)
18. Erling, O., Averbuch, A., Larriba-Pey, J., Chafi, H., Gubichev, A., Prat, A., Pham, M.D., Boncz, P.: The ldbc social network benchmark: Interactive workload. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pp. 619–630 (2015)
19. Eschauzier, R., Taelman, R., Verborgh, R.: Revisiting link prioritization for efficient traversal in structured decentralized environments. In: *International Semantic Web Conference*, pp. 575–593. Springer (2025)
20. Esteves, B., Pandit, H.J., Rodríguez-Doncel, V.: OdrI profile for expressing consent through granular access control policies in solid. In: *2021 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 298–306. IEEE (2021)

21. Fielding, R., Nottingham, M., Reschke, J.: Rfc 9111: Http caching (2022)
22. Folz, P., Skaf-Molli, H., Molli, P.: Cyclades: a decentralized cache for triple pattern fragments. In: European semantic web conference. pp. 455–469. Springer (2016)
23. Hamill, L., Gilbert, G.: Social circles: A simple structure for agent-based social network models. *Journal of Artificial Societies and Social Simulation* **12**(2) (2009)
24. Hanski, J., Taelman, R., Verborgh, R.: Observations on bloom filters for traversal-based query execution over solid pods. In: European Semantic Web Conference. pp. 228–233. Springer (2024)
25. Hanski, J., Van Braeckel, S., Taelman, R., Verborgh, R.: Link traversal over decentralised environments using restart-based query planning. In: International Conference on Web Engineering (2025)
26. Hartig, O.: How caching improves efficiency and result completeness for querying linked data. In: LDOW (2011)
27. Hartig, O.: Zero-knowledge query planning for an iterator implementation of link traversal based query execution. In: Extended Semantic Web Conference. pp. 154–169. Springer (2011)
28. Hartig, O., Bizer, C., Freytag, J.C.: Executing sparql queries over the web of linked data. In: International semantic web conference. pp. 293–309. Springer (2009)
29. Hartig, O., Özsu, M.T.: Walking without a map: Ranking-based traversal for querying linked data. In: International Semantic Web Conference. pp. 305–324. Springer (2016)
30. Heling, L., Acosta, M.: Federated sparql query processing over heterogeneous linked data fragments. In: Proceedings of the ACM Web Conference 2022. pp. 1047–1057 (2022)
31. Kalaitzakis, A., Papadakis, H., Fragopoulou, P.: Evolution of user activity and community formation in an online social network. In: Proceedings of the 2012 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining. pp. 1315–1320. IEEE (2012)
32. Lorey, J., Naumann, F.: Caching and prefetching strategies for sparql queries. In: Extended Semantic Web Conference. pp. 46–65. Springer (2013)
33. Malyshev, S., Krötzsch, M., González, L., Gonsior, J., Bielefeldt, A.: Getting the most out of wikidata: Semantic technology usage in wikipedia’s knowledge graph. In: The Semantic Web–ISWC 2018: 17th International Semantic Web Conference, Monterey, CA, USA, October 8–12, 2018, Proceedings, Part II 17. pp. 376–394. Springer (2018)
34. Martin, M., Unbehauen, J., Auer, S.: Improving the performance of semantic web applications with sparql query caching. In: Extended Semantic Web Conference. pp. 304–318. Springer (2010)
35. Meiss, M., Duncan, J., Gonçalves, B., Ramasco, J.J., Menczer, F.: What’s in a session: tracking individual behavior on the web. In: Proceedings of the 20th ACM conference on Hypertext and hypermedia. pp. 173–182. ACM, Torino Italy (Jun 2009). <https://doi.org/10.1145/1557914.1557946>, <https://dl.acm.org/doi/10.1145/1557914.1557946>
36. Mislove, A., Marcon, M., Gummadi, K.P., Druschel, P., Bhattacharjee, B.: Measurement and analysis of online social networks. In: Proceedings of the 7th ACM SIGCOMM conference on Internet measurement. pp. 29–42 (2007)
37. Mitzlaff, F., Atzmueller, M., Benz, D., Hotho, A., Stumme, G.: User-relatedness and community structure in social interaction networks. arXiv preprint arXiv:1309.3888 (2013)

38. Nicholson, H., Chrysogelos, P., Ailamaki, A.: Hpcache: memory-efficient olap through proportional caching revisited. *The VLDB Journal* **33**(6), 1775–1791 (2024)
39. O’Neil, P., O’Neil, E., Chen, X., Revilak, S.: The star schema benchmark and augmented fact table indexing. In: *Technology Conference on Performance Evaluation and Benchmarking*. pp. 237–252. Springer (2009)
40. Padia, A., Finin, T., Joshi, A., et al.: Attribute-based fine grained access control for triple stores. In: *3rd Society, Privacy and the Semantic Web-Policy and Technology workshop, 14th International Semantic Web Conference* (2015)
41. Papailiou, N., Tsoumakos, D., Karras, P., Koziris, N.: Graph-aware, workload-adaptive sparql query caching. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. pp. 1777–1792 (2015)
42. Picalausa, F., Vansummeren, S.: What are real SPARQL queries like? In: *Proceedings of the International Workshop on Semantic Web Information Management*. pp. 1–6. SWIM ’11, Association for Computing Machinery, New York, NY, USA (Jun 2011). <https://doi.org/10.1145/1999299.1999306>, <https://dl.acm.org/doi/10.1145/1999299.1999306>
43. Raman, A., Joglekar, S., Cristofaro, E.D., Sastry, N., Tyson, G.: Challenges in the decentralised web: The mastodon case. In: *Proceedings of the internet measurement conference*. pp. 217–229 (2019)
44. Sadagopan, N., Li, J.: Characterizing typical and atypical user sessions in click-streams. In: *Proceedings of the 17th international conference on World Wide Web*. pp. 885–894 (2008)
45. Saleem, M., Ali, M.I., Hogan, A., Mehmood, Q., Ngomo, A.C.N.: Lsq: the linked sparql queries dataset. In: *International semantic web conference*. pp. 261–269. Springer (2015)
46. Sambra, A.V., Mansour, E., Hawke, S., Zereba, M., Greco, N., Ghanem, A., Zagidulin, D., Aboulmaga, A., Berners-Lee, T.: Solid: a platform for decentralized social applications based on linked data. MIT CSAIL & Qatar Computing Research Institute, Tech. Rep. **2016** (2016)
47. Schmidt, M., Görlitz, O., Haase, P., Ladwig, G., Schwarte, A., Tran, T.: Fedbench: A benchmark suite for federated semantic data query processing. In: *The Semantic Web–ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23–27, 2011, Proceedings, Part I 10*. pp. 585–600. Springer (2011)
48. Schneider, F., Feldmann, A., Krishnamurthy, B., Willinger, W.: Understanding online social network usage from a network perspective. In: *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement*. pp. 35–48 (2009)
49. Schwarte, A., Haase, P., Hose, K., Schenkel, R., Schmidt, M.: Fedx: Optimization techniques for federated query processing on linked data. In: *The Semantic Web–ISWC 2011: 10th International Semantic Web Conference, Bonn, Germany, October 23–27, 2011, Proceedings, Part I 10*. pp. 601–616. Springer (2011)
50. Stadler, C., Saleem, M., Mehmood, Q., Buil-Aranda, C., Dumontier, M., Hogan, A., Ngonga Ngomo, A.C.: Lsq 2.0: A linked dataset of sparql query logs. *Semantic Web* **15**(1), 167–189 (2024)
51. Taelman, R., Van Herwegen, J., Vander Sande, M., Verborgh, R.: Comunica: a modular sparql query engine for the web. In: *International Semantic Web Conference*. pp. 239–255. Springer (2018)
52. Taelman, R., Verborgh, R.: Evaluation of link traversal query execution over decentralized environments with structural assumptions (2023)

53. Taelman, R., Verborgh, R.: Link traversal query processing over decentralized environments with structural assumptions. In: International Semantic Web Conference. pp. 3–22. Springer (2023)
54. Tam, B.E., Taelman, R., Colpaert, P., Verborgh, R.: Opportunities for shape-based optimization of link traversal queries. arXiv preprint arXiv:2407.00998 (2024)
55. Umbrich, J., Hose, K., Karnstedt, M., Harth, A., Polleres, A.: Comparing data summaries for processing live queries over linked data. *World Wide Web* **14**(5), 495–544 (2011)
56. Vögele, C., van Hoorn, A., Schulz, E., Hasselbring, W., Krcmar, H.: Wessbas: extraction of probabilistic workload specifications for load testing and performance prediction—a model-driven approach for session-based application systems. *Software & Systems Modeling* **17**(2), 443–477 (2018)
57. Williams, G.T., Weaver, J.: Enabling fine-grained http caching of sparql query results. In: International Semantic Web Conference. pp. 762–777. Springer (2011)
58. Zhang, W.E., Sheng, Q.Z., Qin, Y., Yao, L., Shemshadi, A., Taylor, K.: Secf: improving sparql querying performance with proactive fetching and caching. In: Proceedings of the 31st Annual ACM Symposium on Applied Computing. pp. 362–367 (2016)
59. Zhang, X., Wang, M., Zhao, B., Liu, R., Zhang, J., Yang, H.: Characterizing robotic and organic query in sparql search sessions. In: Web and Big Data: 4th International Joint Conference, APWeb-WAIM 2020, Tianjin, China, September 18–20, 2020, Proceedings, Part I 4. pp. 270–285. Springer (2020)